```
std::vector<double> tickRound(const std::vector<double> &prices, const
VariableTickTable& tickTable) {
# Implement this
  return {};
}
```

Given:

```
struct Tick final {
 double lowerBound; // inclusive
 double upperBound; // exclusive
 double width;
}
struct VariableTickTable final {
 double cabinetPrice;
 std::vector<Tick> tickRanges;
}
```

A price of $1 would be tick-rounded to $2.00 which is the cabinet price.
A price of $2.25 would not have to be tick-round at all, because it is already on a tick.
A price of $3.27 would be tick-round to $3.30 which is the nearest multiple of $0.05 above $3.00
A price of $15.80 would be tick-rounded to $16.00 which is the nearest multiple of $1.00 above $10.00

Valid ticks are calculated from the lower bound of the tick range, not from $0.00!

STDIN:

```
2.0 # cabinet price
3 # 3 tick ranges
2.0 3.0 0.01  # from $2.00 (inclusive) to $3.00 (exclusive), the tick width is $0.01
3.0 10.0 0.05  # from $3.00 (inclusive) to $10.00 (exclusive), the tick width is $0.05
10.0 inf 1.00  # from $10.00 (inclusive) and upward, the tick width is $1.00
4 # there are 4 test prices
2.25
1.0
3.27
15.8
```

```
#include <vector>
#include <cmath>
#include <limits>

std::vector<double> tickRound(const std::vector<double> &prices, const
VariableTickTable &tickTable) {
  std::vector<double> result;
  result.reserve(prices.size());

  for (double price : prices) {
    // If below cabinet price, round up to cabinet price
    if (price < tickTable.cabinetPrice) {
      result.push_back(tickTable.cabinetPrice);
      continue;
    }

    // Find the appropriate tick range for this price
    const Tick* tickRange = nullptr;
    for (const auto& tick : tickTable.tickRanges) {
      if (price >= tick.lowerBound && price < tick.upperBound) {
        tickRange = &tick;
        break;
      }
    }

    // If no range found (shouldn't happen with valid input), use the last range
    if (!tickRange && !tickTable.tickRanges.empty()) {
      tickRange = &tickTable.tickRanges.back();
    }

    if (!tickRange) {
      result.push_back(price);
      continue;
    }

    // Calculate offset from the lower bound of this tick range
    double offset = price - tickRange->lowerBound;

    // How many tick widths fit in this offset?
    double numTicks = offset / tickRange->width;

    // Round to the nearest tick
    double roundedNumTicks = std::round(numTicks);
```

```
      // Calculate the final rounded price
      double roundedPrice = tickRange->lowerBound + roundedNumTicks * tickRange->width;

      result.push_back(roundedPrice);
    }

    return result;
}
```